

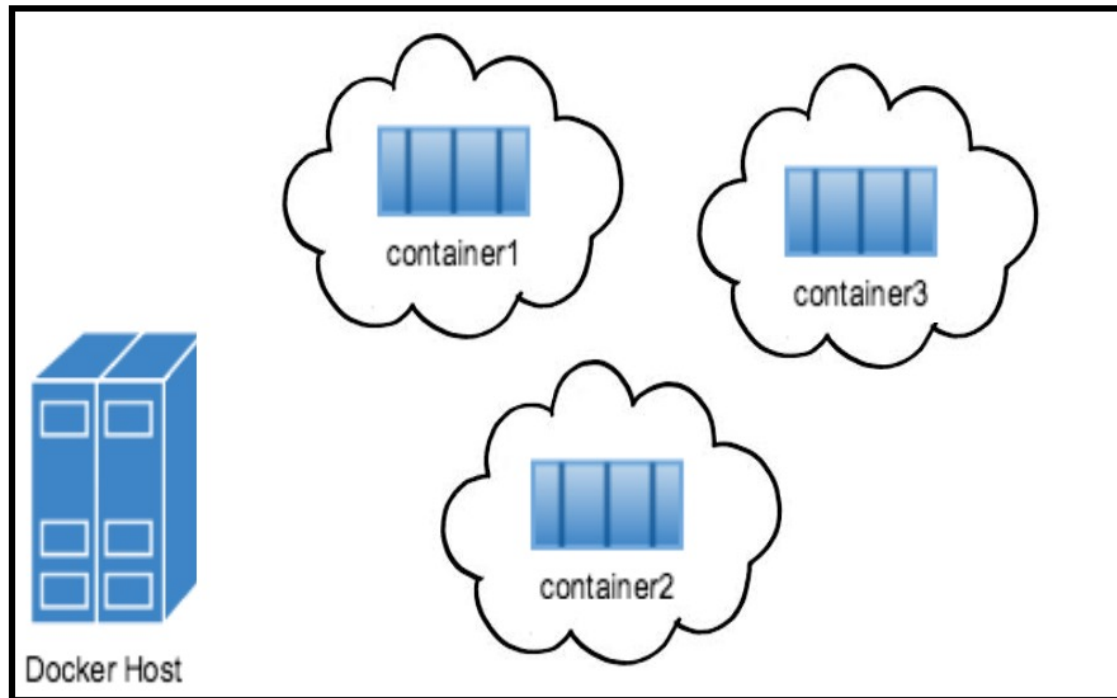


Cloud Specialist
Unità Formativa (UF): Containers Microservizi
Serverless
Docente: Denis Maggiorotto
Titolo argomento: Container networking

Gestione della rete all'interno dei container



None network



None network

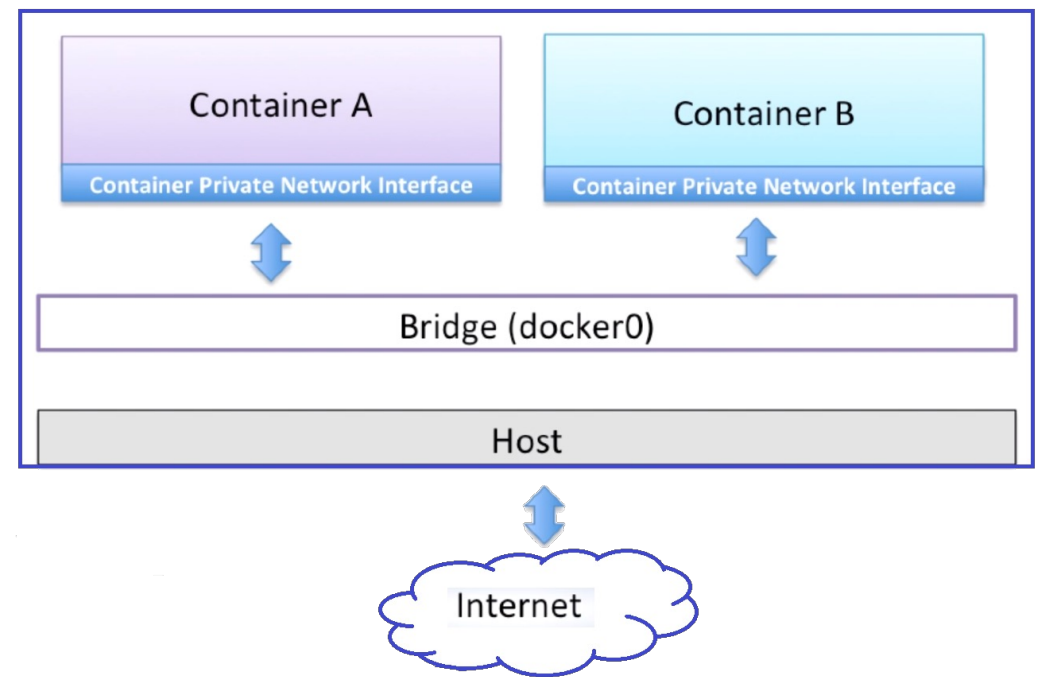
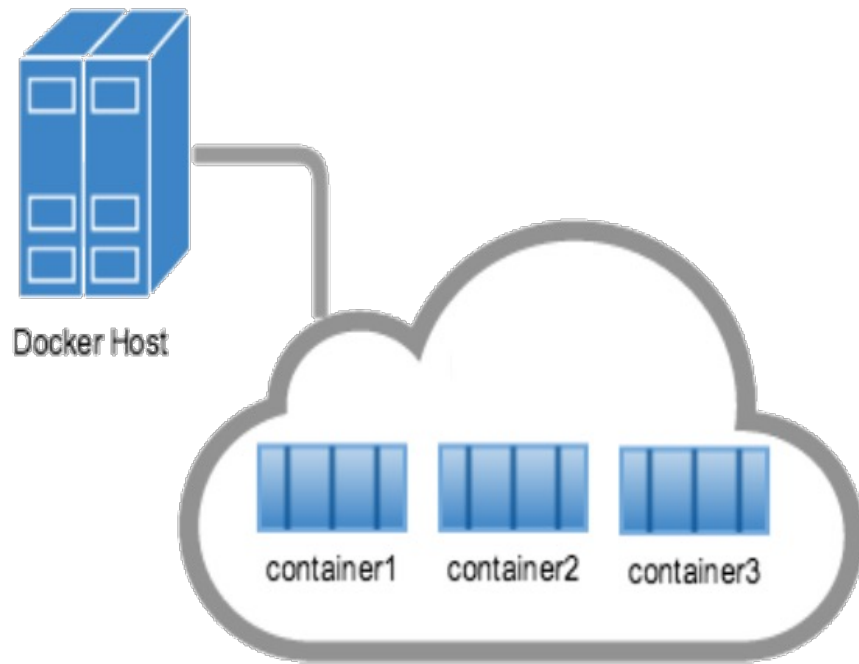
```
$ docker run -d --net none busybox sleep 1000
3a82e2e4537ac9cde52fb69c3a471fd4809817a6165983f0bf5af08d67eebe3

$ docker exec -it
3a82e2e4537ac9cde52fb69c3a471fd4809817a6165983f0bf5af08d67eebe3
/bin/ash

/ # ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8): 56 data bytes
ping: sendto: Network is unreachable

/ # ifconfig
lo          Link encap:Local Loopback
            inet addr:127.0.0.1  Mask:255.0.0.0
            UP LOOPBACK RUNNING  MTU:65536  Metric:1
```

Bridge network



Bridge network

```
$ docker network ls
```

NETWORK ID	NAME	DRIVER	SCOPE
8dae0c4d0380	bridge	bridge	local
22530231e5cd	host	host	local
bd5e44929d16	none	null	local

Bridge network

IP Range:

172.17.0.0 –
172.17.255.255

```
$ docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
3cbd8224e190        bridge             bridge              local
82dd0e241d70        host               host                local
b194d5fd90b6        none              null                local
$ docker network inspect bridge
[
  {
    "Name": "bridge",
    "Id": "3cbd8224e1908396742d7d9ba01fc958657e84e163c0458b3552b6f49391b63b",
    "Created": "2016-12-04T10:38:46.196030628Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": null,
      "Config": [
        {
          "Subnet": "172.17.0.0/16"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Containers": {},
    "Options": {
      "com.docker.network.bridge.default_bridge": "true",
      "com.docker.network.bridge.enable_icc": "true",
      "com.docker.network.bridge.enable_ip_masquerade": "true",
      "com.docker.network.bridge.host_binding_ipv4": "0.0.0.0",
      "com.docker.network.bridge.name": "docker0",
      "com.docker.network.driver.mtu": "1500"
    },
    "Labels": {}
  }
]
```

Bridge network

```
$ docker run -d --name container_1 busybox sleep 1000  
34886a41f5212abd8cd3aa4062479028899e78f32b44103bade6b67d88b76f75
```

```
$ docker exec -it container_1 ifconfig  
eth0      Link encap:Ethernet  HWaddr 02:42:AC:11:00:02  
          inet addr:172.17.0.2  Bcast:172.17.255.255  Mask:255.255.0.0  
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1  
          RX packets:10 errors:0 dropped:0 overruns:0 frame:0  
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0  
          collisions:0 txqueuelen:0  
          RX bytes:796 (796.0 B)  TX bytes:0 (0.0 B)  
  
lo        Link encap:Local Loopback  
          inet addr:127.0.0.1  Mask:255.0.0.0  
          UP LOOPBACK RUNNING  MTU:65536  Metric:1  
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0  
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0  
          collisions:0 txqueuelen:1000  
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
```


Bridge network

```
$ docker run -d --name container_2 busybox sleep 1000
c8c34dad781a7a5778852de2e40fe922f6b4113759b94928d81c4753f7631b9a

$ docker exec -it container_2 ifconfig
eth0      Link encap:Ethernet  HWaddr 02:42:AC:11:00:04
          inet addr:172.17.0.4  Bcast:172.17.255.255  Mask:255.255.0.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:10 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:796 (796.0 B)  TX bytes:0 (0.0 B)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
```

Bridge network

```
$ docker exec -it container_1 ping 172.17.0.4
PING 172.17.0.4 (172.17.0.4): 56 data bytes
64 bytes from 172.17.0.4: seq=0 ttl=64 time=0.069 ms
64 bytes from 172.17.0.4: seq=1 ttl=64 time=0.076 ms
--- 172.17.0.4 ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 0.069/0.072/0.076 ms
```

```
$ docker exec -it container_1 ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8): 56 data bytes
64 bytes from 8.8.8.8: seq=0 ttl=117 time=33.251 ms
--- 8.8.8.8 ping statistics ---
1 packets transmitted, 1 packets received, 0% packet loss
round-trip min/avg/max = 33.251/33.251/33.251 ms
```

Bridge network

```
$ docker network create --driver bridge my_bridge_network
3d84cfb644d945ec14b1c9ed7f9e9fde10ae66edf41ce26d39fb60f16e00d16f

$ docker network inspect my_bridge_network
[
  {
    "Name": "my_bridge_network",
    "Id":
"3d84cfb644d945ec14b1c9ed7f9e9fde10ae66edf41ce26d39fb60f16e00d16f",
    "Created": "2019-02-08T16:40:38.916055692Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": {},
      "Config": [
        {
          "Subnet": "172.19.0.0/16",
          "Gateway": "172.19.0.1"
        }
      ]
    }
  }
]
```

Bridge network

```
$ docker run -d --name container_3 --net my_bridge_network busybox sleep 1000
Dee203a2804a63935bea37f53d51f1b35b8e769124ca302351c4d192e1c94187

$ docker exec -it container_3 ifconfig
eth0      Link encap:Ethernet  HWaddr 02:42:AC:13:00:02
          inet addr:172.19.0.2  Bcast:172.19.255.255  Mask:255.255.0.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:16 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:1312 (1.2 KiB)  TX bytes:0 (0.0 B)

...

$ docker start container_1
container_1

$ docker exec -it container_1 ifconfig
...
eth0      Link encap:Ethernet  HWaddr 02:42:AC:11:00:02
          inet addr:172.17.0.2  Bcast:172.17.255.255  Mask:255.255.0.0
...

$ docker exec -it container_3 ping 172.17.0.2
PING 172.17.0.2 (172.17.0.2): 56 data bytes

...
--- 172.17.0.2 ping statistics ---
424 packets transmitted, 0 packets received, 100% packet loss
```

Bridge network

```
$ docker network connect bridge container_3

$ docker exec -it container_3 ifconfig
eth0      Link encap:Ethernet  HWaddr 02:42:AC:13:00:02
          inet addr:172.19.0.2  Bcast:172.19.255.255  Mask:255.255.0.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:36 errors:0 dropped:0 overruns:0 frame:0
          TX packets:434 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:2432 (2.3 KiB)  TX bytes:41972 (40.9 KiB)

eth1      Link encap:Ethernet  HWaddr 02:42:AC:11:00:04
          inet addr:172.17.0.4  Bcast:172.17.255.255  Mask:255.255.0.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:9 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:726 (726.0 B)  TX bytes:0 (0.0 B)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

$ docker exec -it container_3 ping 172.17.0.2
PING 172.17.0.2 (172.17.0.2): 56 data bytes
64 bytes from 172.17.0.2: seq=0 ttl=64 time=0.489 ms
64 bytes from 172.17.0.2: seq=1 ttl=64 time=0.116 ms

--- 172.17.0.2 ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 0.082/0.160/0.489 ms

$ docker network disconnect bridge container_3
```

Host network

```
$ docker run -d --name container_6 --net host busybox sleep 1000
Baad65ed37ad53ab705bf9f9bbf50c779736bbda128859111f28fad24473421b
```

```
$ docker exec -it container_6 ifconfig
br-3d84cfb644d9 Link encap:Ethernet HWaddr 02:42:4C:D0:FD:D5
    inet addr:172.19.0.1 Bcast:172.19.255.255 Mask:255.255.0.0
    inet6 addr: fe80::42:4cff:fed0:fdd5/64 Scope:Link
    UP BROADCAST MULTICAST MTU:1500 Metric:1
    RX packets:434 errors:0 dropped:0 overruns:0 frame:0
    TX packets:26 errors:0 dropped:0 overruns:0 carrier:0
    collisions:0 txqueuelen:0
    RX bytes:35896 (35.0 KiB) TX bytes:1676 (1.6 KiB)
br-6b74b77377ec Link encap:Ethernet HWaddr 02:42:51:A1:81:6B
    inet addr:172.18.0.1 Bcast:172.18.255.255 Mask:255.255.0.0
    UP BROADCAST MULTICAST MTU:1500 Metric:1
    RX packets:0 errors:0 dropped:0 overruns:0 frame:0
    TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
    collisions:0 txqueuelen:0
    RX bytes:0 (0.0 B) TX bytes:0 (0.0 B)
```

Host network

```
docker0  Link encap:Ethernet  HWaddr 02:42:25:25:B1:24
          inet addr:172.17.0.1  Bcast:172.17.255.255  Mask:255.255.0.0
          inet6 addr: fe80::42:25ff:fe25:b124/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:103 errors:0 dropped:0 overruns:0 frame:0
          TX packets:220 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:128663 (125.6 KiB)  TX bytes:18397 (17.9 KiB)

eth0     Link encap:Ethernet  HWaddr 08:00:27:2E:6E:CE
          inet addr:10.0.2.15  Bcast:10.0.2.255  Mask:255.255.255.0
          inet6 addr: fe80::a00:27ff:fe2e:6ece/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:146537 errors:0 dropped:0 overruns:0 frame:0
          TX packets:10959 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:204339347 (194.8 MiB)  TX bytes:1083768 (1.0 MiB)
```

LAB

https://github.com/sunnyvale-academy/ITS-ICT_Containers

Lab 08 – Docker networking

Assignment 02 - Multi-network Wordpress installation

Esposizione delle porte di un container



Esposizione di porte da un container

Prendiamo come esempio i seguenti container:

httpd (Apache)

```
$ docker run --network bridge --name container_1 httpd:latest  
...
```

MySQL

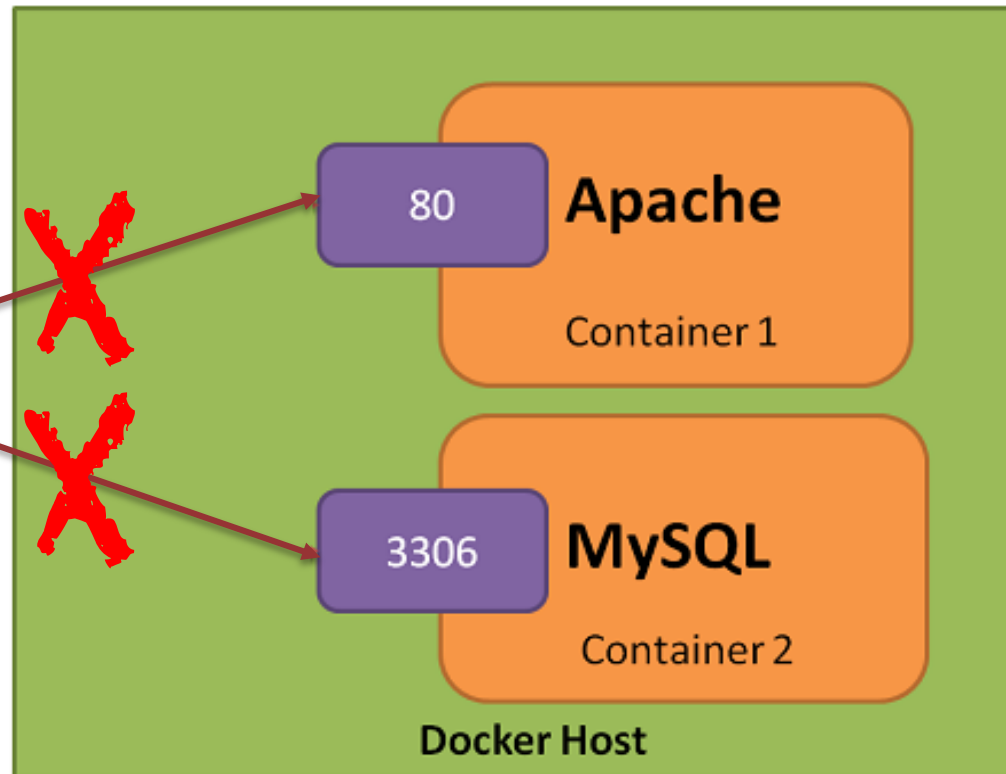
```
$ docker run --network bridge --name container_2 mysql:latest  
...
```

Esposizione di porte da un container

Bridge network

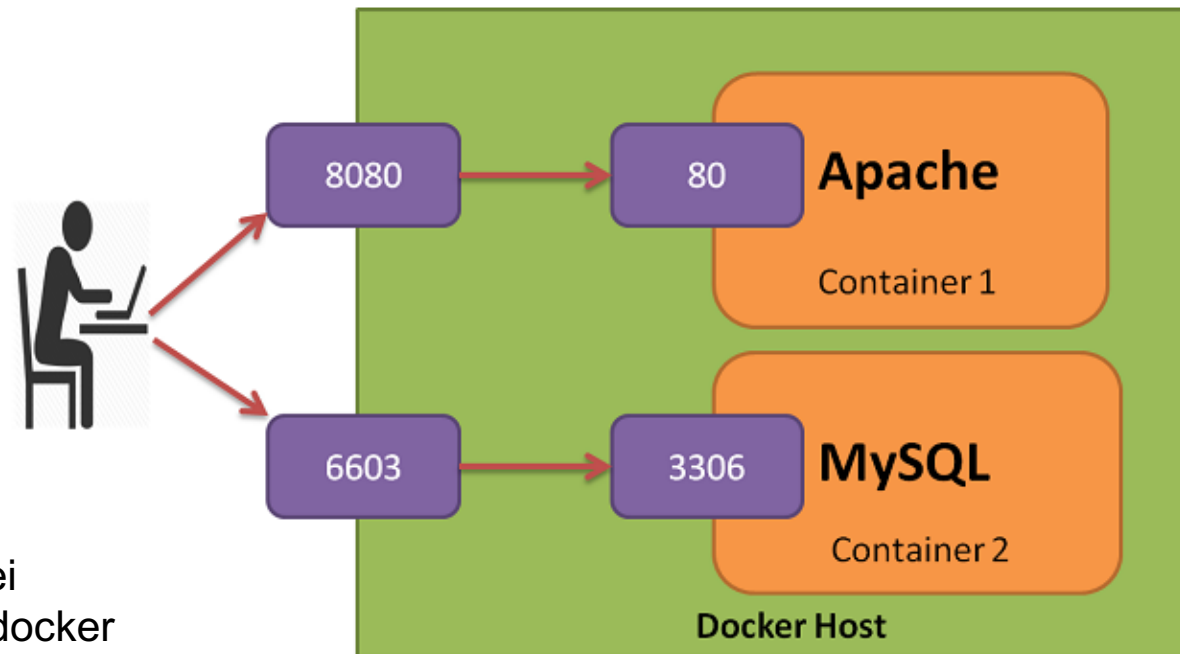


I container creati con una network di tipo «bridge», di default non espongono le porte di rete sul docker host



Esposizione di porte da un container

Bridge network



Per poter accedere ai servizi, occorre rimappare le porte dei container su porte libere del docker host

Esposizione di porte da un container

Per farlo, usiamo il parametro **-p** seguito da una coppia di porte separate dal carattere :

httpd (Apache)

Porta su docker host : porta del container

```
$ docker run -p 8080:80 --network bridge --name container_1 httpd:latest  
...
```

MySQL

Porta su docker host : porta del container

```
$ docker run -p 6603:3306 --network bridge --name container_2 mysql:latest  
...
```

Esposizione di porte da un container

Flag value	Description
<code>-p 8080:80</code>	Map TCP port 80 in the container to port 8080 on the Docker host.
<code>-p 192.168.1.100:8080:80</code>	Map TCP port 80 in the container to port 8080 on the Docker host for connections to host IP 192.168.1.100.
<code>-p 8080:80/udp</code>	Map UDP port 80 in the container to port 8080 on the Docker host.
<code>-p 8080:80/tcp -p 8080:80/udp</code>	Map TCP port 80 in the container to TCP port 8080 on the Docker host, and map UDP port 80 in the container to UDP port 8080 on the Docker host.

Esposizione di porte da un container

Prendiamo come esempio i seguenti container:

httpd (Apache)

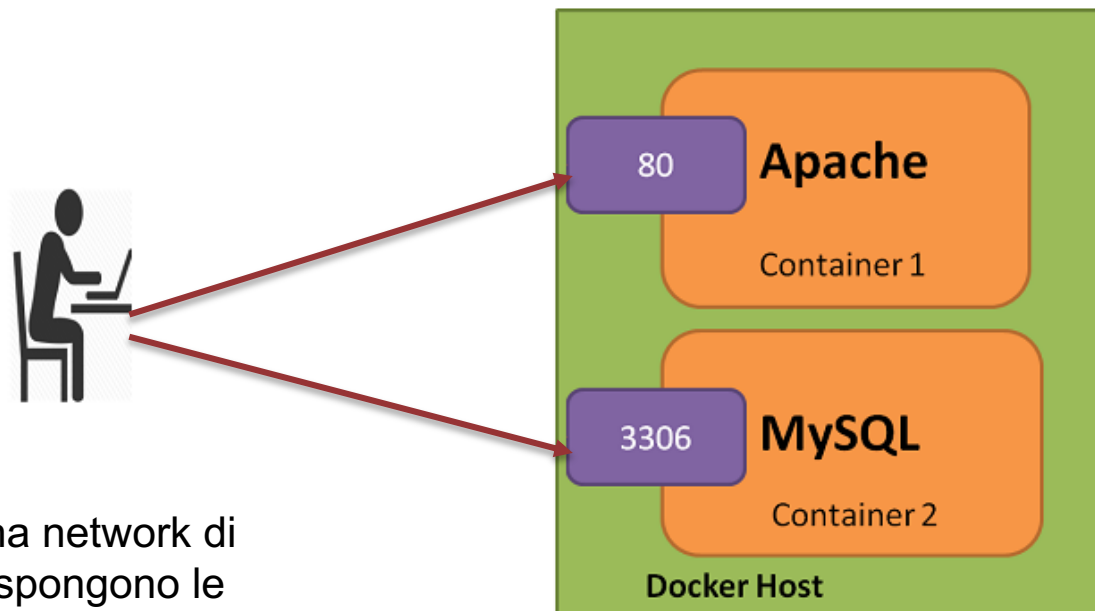
```
$ docker run --network host --name container_1 httpd:latest  
...
```

MySQL

```
$ docker run --network host --name container_2 mysql:latest  
...
```

Esposizione di porte da un container

Host network



I container creati con una network di tipo «host», di default espongono le porte di rete sul docker host

Esposizione di porte da un container

Prendiamo come esempio i seguenti container:

httpd (Apache)

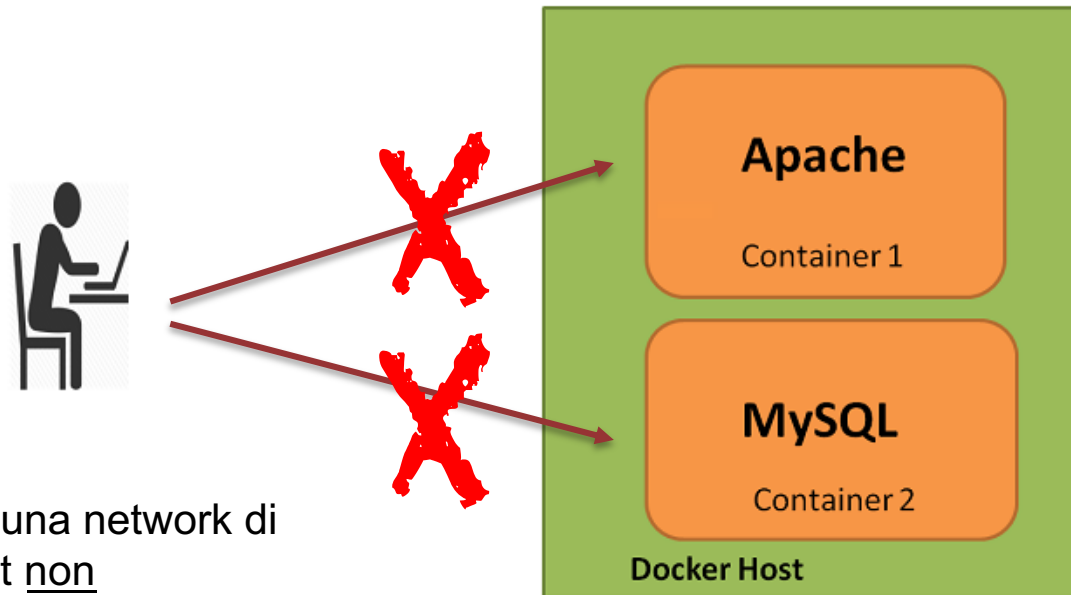
```
$ docker run --network none --name container_1 httpd:latest  
...
```

MySQL

```
$ docker run --network none --name container_2 mysql:latest  
...
```

Esposizione di porte da un container

None network



I container creati con una network di tipo «none», di default non espongono servizi sull'esterno

LAB

https://github.com/sunnyvale-academy/ITS-ICT_Containers

Lab 09 – Containers ports